# Architecture

## Approach

The project is a local desktop application with three cooperating parts:

1. **Capture engine** (`src/capture/`) — reads a PCAP/PCAPNG file and produces a normalized capture: TCP sessions, reassembled directional streams, and per-byte provenance.
2. **Protocol and hypothesis engine** (`src/protocol/`, `src/hypothesis/`) — frames a directional stream into messages, applies declarative interpretation rules, classifies the results, verifies rules on a corpus, finds counterexamples, and versions rules and results.
3. **Project, report, and UI** — `src/project/` (the portable investigation), `src/report/` (the report renderers), and `src/ui/` (the PySide6 interface).

The interface between the parts is fixed by the JSON contracts in `docs/CONTRACT.md` and the schemas in `docs/schemas/`. No part reaches into another's data model; they exchange the contracted JSON and plain dictionaries.

Three principles shape every module:

- **Observation is separate from hypothesis.** The capture engine reports bytes and where they came from. It never says what a byte means.
- **Counterexamples are required.** A rule that matches examples is reported as confirmed in the tested scope, and the data that contradicts it is kept.
- **Reproducibility.** Given the same input, every step produces the same output, and a project carries relative paths and content digests so it opens unchanged on another machine.

## Capture engine

### Library choice

PCAP and PCAPNG reading uses **dpkt 1.9.8**.

- dpkt reads both classic pcap (`dpkt.pcap.Reader`) and pcapng (`dpkt.pcapng.Reader`) without external processes.
- PyShark was rejected because it shells out to `tshark`, which would have to be present on the target machine; the reference environment does not guarantee it.
- scapy is kept as a mental fallback but is not a dependency: it is heavier and slower for the reference workload of up to 250k packets.

IPv6 and several link-layer types (raw IP, Linux cooked capture, Linux loopback, BSD cooked) are parsed; an unsupported link type or a malformed header is reported as a diagnostic instead of raising, so a mixed capture still yields the TCP sessions that can be read.

### Modules

- `parser.py` — `read_capture(path) -> Iterator[Packet]`. Yields TCP packets with a monotonic `index`, timestamp, endpoints, sequence numbers, flags, and payload. Non-TCP packets are skipped with a diagnostic where relevant. An unsupported link type is reported as a diagnostic; IPv6, including the extension-header chain, is parsed. A fragmented datagram is handed to `fragment.py` and

yields one packet once its fragments are complete. A damaged but recognizable file is read as far as possible and the rest is reported as diagnostics; only a non-capture raises.

- `fragment.py` — `FragmentReassembler` collects IPv4 and IPv6 fragments and returns a datagram when every byte is present. IPv4 keeps the first arrival on an overlap and reports it; IPv6 discards the datagram on any overlap, as RFC 5722 requires. A datagram that never completes is reported as `ip_fragment_incomplete` and no bytes are guessed.
- `session.py` — `build_sessions(packets) -> list[Session]`. Groups packets into TCP connection instances. A new session is started on SYN without ACK, on a new tuple after a FIN/RST, and on any reuse of an endpoint pair after the previous instance ended.
- `reassembly.py` — `reassemble(session, direction, packets) -> DirectionalStream`. Orders payload by sequence number, deduplicates identical retransmissions, buffers out-of-order segments, marks gaps explicitly, records contradictory overlaps as ambiguity, and handles the 32-bit sequence wrap.
- `provenance.py` — `Range` and helpers mapping stream offsets to source packets.
- `export.py` — `export_capture(...)` writes the contract JSON, streaming so the whole file is not held in memory.
- `export_wireshark.py` — writes one packet per session direction carrying the reassembled bytes, for inspection in Wireshark.
- `streaming.py` — `process_streaming(...)` releases a closed session from memory while walking a large capture.
- `cli.py` — `python -m src.capture.cli --pcap ... --out ...`. `--verbose` logs read, session and export progress to stderr; without it the command prints only the JSON summary.

## Project and report

- `src/project/manifest.py` — the on-disk manifest model and its validation.
- `src/project/project.py` — `Project`: create, open, `add_capture`, `export`, `import_`. Every manifest path is relative and each capture carries its sha256, so the directory can be moved and reopened; import rejects a tampered archive.
- `src/report/markdown.py` and `src/report/html.py` — render an investigation dictionary to a Markdown report and to a self-contained HTML page styled per `docs/STYLE.md`.
- `src/report/model.py` — the investigation dictionary shape shared by both renderers and produced by the protocol stage.

## Formats

See `docs/CONTRACT.md`. The three formats are:

1. **Normalized capture** — capture engine output, protocol and UI input.
2. **Rule application result** — protocol engine output, UI input.
3. **Project manifest** — the index of an on-disk investigation.

## Dependencies

| Package | Version | Why |
|---|---|---|
| `dpkt` | 1.9.8 | Reads pcap and pcapng in pure Python, no external process. |
| `PyYAML` | 6.0.2 | Interpretation rules may be written as YAML as well as JSON. |
| `PySide6` | 6.8.1 | The desktop interface. |
| `pytest` | 8.3.4 | Tests (development extra). |
| `jsonschema` | 4.23.0 | Validates the exports against the schemas (development extra). |

## Limits and what is not implemented

- **IPv6** is parsed. TCP sessions over IPv6 yield streams like IPv4, and IPv6 TCP checksums are verified with `--verify-checksums`. IPv6 fragmentation is reassembled; an overlap causes the whole datagram to be discarded, as RFC 5722 requires, and the conflict is reported.
- **IP fragmentation** is reassembled for both versions. Fragments of one datagram may be separated by unrelated traffic and delivered out of order. An overlap is reported for IPv4 (first arrival kept) and discards the datagram for IPv6. A datagram that never completes is reported as `ip_fragment_incomplete`; its bytes are not guessed.
- **Encrypted traffic** is not decrypted. TLS and anything else above TCP appears as opaque bytes; the byte-level tools still work on it.
- **Only Ethernet, Linux loopback, and BSD cooked** link types are understood. Others are reported as `unsupported_linktype` and skipped.
- **Checksum validation is off by default**, because of checksum offloading on the sending host; it can be turned on when the capture is known to be intact.
- **Conflicting overlaps are not resolved.** The first bytes are kept and the contradiction is reported as `ambiguity`, rather than one version being silently chosen.

## Performance

The measured reference workload is in `docs/BENCHMARK.md`: 95.66 MiB input, 250k packets, 1000 sessions, in 11.56 s at 354 MiB peak in memory and 11.22 s at 224 MiB with streaming, producing 122.44 MiB of JSON. Both are well inside the 5 minute and 4 GiB limits of the case.